

DAY 18 — PostgreSQL Transactions & ACID

Today we move from:

SQL

↓

Database design

↓

Indexes

↓

Transactions

This is where PostgreSQL starts becoming a **production system**, not just a place to store/query data.

The key question today is:

How do we make multiple database operations behave as one reliable unit of work?

For a GenAI/RAG engineer, this matters when you're doing things like:

Create document

↓

Create chunks

↓

Create metadata

↓

Create ingestion status

If step 3 fails, you often don't want half the operation committed.

1. What Is a Transaction?

A transaction is a group of database operations treated as one logical unit.

Example:

```
BEGIN;
```

```
INSERT INTO documents (...);
```

```
INSERT INTO document_chunks (...);
```

```
INSERT INTO ingestion_jobs (...);
```

```
COMMIT;
```

Conceptually:

BEGIN

|

|— operation 1

|— operation 2

|— operation 3

|



COMMIT

|



All changes become permanent

If something goes wrong:

ROLLBACK;

then the transaction's changes are undone.

2. Why Transactions Exist

Imagine a banking-style operation:

Account A

₹10,000

Account B

₹5,000

Transfer:

A - ₹1,000

B + ₹1,000

You don't want:

A - ₹1,000

B + ₹1,000 failed

because money disappeared.

You want:

BEGIN

A = A - 1000

B = B + 1000

COMMIT

Either:

both succeed

or:

neither succeeds

That's the fundamental purpose of transactions.

3. BEGIN

Start a transaction:

```
BEGIN;
```

Then perform operations:

```
INSERT INTO documents (  
id,  
user_id,  
title  
)  
VALUES (  
101,  
1,  
'RAG Fundamentals'  
);
```

Then:

```
COMMIT;
```

4. COMMIT

COMMIT makes the transaction's changes permanent.

```
BEGIN;
```

```
INSERT INTO users (  
id,  
name  
)  
VALUES (  
1,  
'Aarav'  
);
```

```
COMMIT;
```

After commit, the change is persisted as part of the completed transaction.

5. ROLLBACK

Suppose:

```
BEGIN;
```

```
    INSERT INTO users (  
id,  
name  
)  
    VALUES (  
1,  
'Aarav'  
);
```

```
ROLLBACK;
```

The insert is undone.

Conceptually:

Before BEGIN

↓

No Aarav

```
INSERT
```

↓

Aarav exists inside transaction

```
ROLLBACK
```

↓

Aarav no longer exists

6. The Most Important Mental Model

Think:

```
BEGIN
```

↓

temporary transaction state

↓

multiple operations

↓

```
COMMIT
```

↓

durable result

or:

BEGIN

↓

multiple operations

↓

ROLLBACK

↓

undo transaction changes

7. ACID

You must know:

A → Atomicity

C → Consistency

I → Isolation

D → Durability

These are the four fundamental transaction properties.

8. A — Atomicity

Atomicity means:

A transaction happens completely or not at all.

Suppose RAG ingestion does:

1. Insert document

2. Insert chunks

3. Insert metadata

4. Update ingestion status

If step 3 fails:

Without a transaction:

document

chunks

metadata

status

You may have inconsistent partial data.

With a transaction:

BEGIN

document

chunks

metadata

ROLLBACK

Everything undone

That's atomicity.

9. C — Consistency

Consistency means:

A transaction takes the database from one valid state to another valid state, respecting defined constraints and rules.

Suppose:

page_count INTEGER

CHECK (page_count > 0)

and:

INSERT INTO documents (

page_count

)

VALUES (-10);

The constraint rejects it.

The transaction cannot successfully commit that invalid state.

Consistency is closely connected to:

PRIMARY KEY

FOREIGN KEY

UNIQUE

CHECK

NOT NULL

which you learned on Day 16.

10. I — Isolation

Isolation concerns concurrent transactions.

Imagine:

Transaction A

Transaction B

running simultaneously.

Without appropriate isolation, one transaction might observe data changes from another transaction in problematic ways.

PostgreSQL provides transaction isolation mechanisms to control what concurrent transactions can observe.

We'll go deep into this today.

11. D — Durability

After:

COMMIT;

the committed transaction should survive failures according to PostgreSQL's durability guarantees and configuration.

Conceptually:

COMMIT

↓

database acknowledges success

↓

committed state survives crash/restart

PostgreSQL uses mechanisms including:

WAL

Write-Ahead Logging

to provide durability and crash recovery.

You don't need to master WAL internals today.

Just understand:

PostgreSQL records changes in a way that supports recovery after failures.

12. Transaction Example — RAG Ingestion

Suppose a document ingestion pipeline does:

Upload document

↓

Create document row

↓

Create chunks

↓

Create embedding records

↓

Mark document processed

A simplified transaction:

```
BEGIN;
```

```
    INSERT INTO documents (  
id,  
user_id,  
title,  
status  
)  
    VALUES (  
101,  
1,  
'RAG Guide',  
'processing'  
);
```

```
    INSERT INTO document_chunks (  
document_id,  
chunk_number,  
content  
)  
    VALUES  
(  
101,  
1,  
'Introduction to RAG'  
)  
(  
101,  
2,  
'Retrieval systems'  
);
```

```
    INSERT INTO embeddings (  
chunk_id,  
model,  
status  
)
```

```
VALUES
(
1,
'embedding-model',
'processing'
);

UPDATE documents
SET status = 'processed'
WHERE id = 101;

COMMIT;
If a critical operation fails:
ROLLBACK;
```

13. Savepoints

Sometimes you don't want to rollback the entire transaction.

PostgreSQL supports:

```
SAVEPOINT
```

Example:

```
BEGIN;
```

```
INSERT INTO users (
id,
name
)
VALUES (
1,
'Aarav'
);
```

```
SAVEPOINT user_insert;
```

```
INSERT INTO documents (
id,
user_id,
title
)
VALUES (
```

101,

1,

'RAG Guide'

);

Now suppose you want to undo only changes after the savepoint:

```
ROLLBACK TO SAVEPOINT user_insert;
```

Then continue:

```
COMMIT;
```

Conceptually:

```
BEGIN
```

```
|
```

```
|— Insert user
```

```
|
```

```
|— SAVEPOINT
```

```
|
```

```
|— Insert document
```

```
|
```

```
|— problem
```

```
|
```

```
|— ROLLBACK TO SAVEPOINT
```

```
↓
```

document undone

user preserved

```
↓
```

```
COMMIT
```

14. SAVEPOINT Use Case

Suppose you're processing many optional records.

Transaction

```
|— required operation
```

```
|— optional operation A
```

```
|— optional operation B
```

```
|— required final operation
```

You might use savepoints around a section where you want partial rollback.

But don't overuse savepoints.

They're a tool for specific transactional workflows, not a replacement for good transaction design.

15. Transactions and Errors

Suppose:

```
BEGIN;
```

```
INSERT INTO users (  
id,  
name  
)  
VALUES (  
1,  
'Aarav'  
);
```

```
INSERT INTO users (  
id,  
name  
)  
VALUES (  
1,  
'Emma'  
);
```

The second insert violates the primary key.

After a statement error, PostgreSQL puts the transaction into an **aborted state**.

You generally need:

```
ROLLBACK;
```

before starting a fresh transaction.

This is an important PostgreSQL behavior.

16. Transaction State

Think:

```
BEGIN
```

↓

```
ACTIVE
```

↓

```
statement error
```

↓

ABORTED TRANSACTION

↓

ROLLBACK

↓

ready again

A common beginner mistake is:

"One query failed, so I'll just run another query."

Inside the failed transaction, that generally won't work until you rollback.

17. Autocommit

Most database clients operate with some form of **autocommit** behavior unless you explicitly start a transaction.

For example:

```
INSERT INTO users ...
```

may be committed automatically by the client/session if you're not inside an explicit transaction.

Explicit:

```
BEGIN;
```

```
INSERT ...
```

```
COMMIT;
```

gives you transaction control.

18. Transactions in Python

This becomes extremely important for your FastAPI/Data Engineering work.

Conceptually:

```
connection = ...
```

```
try:
```

```
connection.execute(...)
```

```
connection.execute(...)
```

```
connection.commit()
```

```
except Exception:
```

```
connection.rollback()
```

```
raise
```

The exact API depends on whether you're using:

psycopg

SQLAlchemy

asyncpg

You'll learn database access patterns later.

The principle remains:

try

↓

database operations

↓

commit

except

↓

rollback

19. Transaction Boundaries

One of the most important production design decisions is:

Where should the transaction begin and end?

Too small:

transaction

↓

one tiny operation

↓

commit

transaction

↓

another operation

↓

commit

You may lose atomicity across related operations.

Too large:

BEGIN

↓

huge amount of work

↓

network calls

↓

LLM API call



embedding API call



more processing



COMMIT

This can hold database resources for too long.

20. Very Important RAG Rule

Avoid keeping a PostgreSQL transaction open while waiting for an external API if you don't need to.

Bad conceptual design:

BEGIN



insert document



call LLM API



wait 5 seconds



call embedding API



wait 10 seconds



insert chunks



COMMIT

You're keeping a database transaction open while external services are running.

Better architecture may separate stages:

DB transaction



create ingestion job



COMMIT

external processing



LLM / embedding API

DB transaction

↓

store results

↓

COMMIT

This is a major production engineering concept.

21. Isolation Levels

PostgreSQL supports:

READ COMMITTED

REPEATABLE READ

SERIALIZABLE

PostgreSQL also accepts READ UNCOMMITTED, but PostgreSQL's behavior effectively treats it like READ COMMITTED.

For your career, focus on:

READ COMMITTED

REPEATABLE READ

SERIALIZABLE

22. READ COMMITTED

This is PostgreSQL's default isolation level.

The key idea:

Each statement sees data committed before that statement began.

Example:

Transaction A

Transaction B

Transaction A:

BEGIN;

SELECT balance

FROM accounts

WHERE id = 1;

Transaction B may commit changes.

A later statement in A can see newly committed data that wasn't visible to the earlier statement.

So within one transaction:

Statement 1

↓

snapshot

Transaction B commits

Statement 2

↓

can see newer committed state

This is a crucial property of PostgreSQL's default isolation.

23. REPEATABLE READ

At:

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
```

the transaction works from a consistent snapshot.

Conceptually:

```
BEGIN
```

↓

snapshot established

↓

statement 1

↓

other transaction commits

↓

statement 2

↓

same transaction snapshot

This provides stronger consistency for repeated reads within a transaction.

24. SERIALIZABLE

Strongest standard level among PostgreSQL's listed levels:

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

The goal is:

Concurrent transactions behave as though they executed serially.

This can result in serialization failures.

So your application must be prepared to:

retry transaction

when PostgreSQL determines concurrent operations cannot safely be serialized.

25. Isolation Problems

Know these concepts.

Dirty Read

Transaction B reads data written by A before A commits.

PostgreSQL does **not** allow ordinary dirty reads.

Non-repeatable Read

Within one transaction:

read row

↓

another transaction changes + commits

↓

read same row again

↓

different value

Possible under:

READ COMMITTED

Phantom Read

A repeated query returns a different set of rows because another transaction inserted/deleted matching rows.

Isolation levels determine which anomalies are possible.

Don't just memorize names—understand the scenario.

26. Lost Update

Consider:

balance = ₹1000

Two transactions both read:

₹1000

Transaction A:

$1000 - 200 = 800$

Transaction B:

$1000 - 300 = 700$

If both overwrite based on stale values, one update can effectively overwrite the other.

This is a concurrency problem.

We'll address this using:

row locking

transactions

atomic updates

isolation

27. Atomic UPDATE

Often you can avoid a read-modify-write race.

Instead of:

```
SELECT balance
FROM accounts
WHERE id = 1;
```

then application code:

```
balance = balance - 100
```

then:

```
UPDATE accounts
SET balance = ...
WHERE id = 1;
```

prefer:

```
UPDATE accounts
SET balance = balance - 100
WHERE id = 1;
```

This lets PostgreSQL perform the arithmetic atomically at the statement level.

28. Row-Level Locking

PostgreSQL provides:

```
SELECT ... FOR UPDATE;
```

Example:

```
BEGIN;
```

```
SELECT balance
FROM accounts
WHERE id = 1
```

```
FOR UPDATE;
```

The selected row is locked against conflicting updates until the transaction ends.

Conceptually:

Transaction A

↓

SELECT ... FOR UPDATE

↓

row locked

↓

Transaction B tries conflicting operation

↓

waits

This is useful when you need to:

read

then

make a dependent decision

then

update

without another transaction changing the row between those steps.

29. RAG Example — Prevent Double Processing

Suppose:

ingestion_jobs

contains:

job_id | status

-----+-----

101 | pending

Two workers might both try:

"Claim job 101"

You don't want:

Worker A → job 101

Worker B → job 101

both processing it.

A locking strategy can help coordinate workers.

Conceptually:

BEGIN;

SELECT *

FROM ingestion_jobs

WHERE status = 'pending'

ORDER BY job_id

LIMIT 1

FOR UPDATE;

```
UPDATE ingestion_jobs
SET status = 'processing'
WHERE job_id = ...;
```

COMMIT;

In production job queues, you may also encounter:

FOR UPDATE SKIP LOCKED

which lets workers skip rows currently locked by another worker.

This is highly relevant to Data Engineering.

30. SKIP LOCKED

Imagine:

Jobs:

101 pending

102 pending

103 pending

Worker A locks:

101

Worker B doesn't need to wait for 101.

With:

FOR UPDATE SKIP LOCKED

it can move to:

102

This is useful for concurrent work queues.

31. Transaction + Constraint

Transactions and constraints work together.

Suppose:

```
BEGIN;
```

```
INSERT INTO documents (
id,
user_id,
```

```
page_count
```

```
)
```

```
VALUES (
```

```
101,
```

```
9999,
```

```
20
```

```
);
```

If user 9999 doesn't exist:

```
FOREIGN KEY violation
```

The statement fails.

The transaction is now in an error state.

You need to:

```
ROLLBACK;
```

or handle the failure appropriately.

32. Transaction + Index

Indexes also participate in transactions.

If:

```
BEGIN;
```

```
INSERT INTO documents (...);
```

```
ROLLBACK;
```

the row insertion is rolled back, and the corresponding index changes are also rolled back.

This is another reason transactions are more than "just grouping queries."

33. Deadlocks

Two transactions can wait for each other.

Example:

Transaction A

locks Row 1

↓

tries to lock Row 2

Transaction B

locks Row 2

↓

tries to lock Row 1

Now:

A waits for B

B waits for A

That's a deadlock.

PostgreSQL detects deadlocks and aborts one transaction.

34. Preventing Deadlocks

One important strategy:

Acquire locks in a consistent order.

For example:

Always lock smaller account ID first.

Instead of:

sometimes

$B \rightarrow A$

use:

smaller ID $\rightarrow$ larger ID

consistently.

This reduces deadlock risk.

35. Transaction Design Rules

For production PostgreSQL:

Good

BEGIN

↓

short database work

↓

COMMIT

Dangerous

BEGIN

↓

database work

↓

HTTP request

↓

LLM request

↓

wait

↓

more processing

↓

COMMIT

Keep transactions:

as short as practical while preserving the required atomicity.

DAY 18 CHALLENGES

No answers yet.

Solve them yourself.

Challenge 1 — Basic Transaction

Write a transaction that:

1. Inserts a document
2. Inserts two chunks for that document
3. Commits everything

Use:

documents

document_chunks

Assume the tables already exist.

Challenge 2 — Rollback

Write a transaction where:

1. A new document is inserted
2. Two chunks are inserted
3. Something goes wrong
4. Everything is rolled back

Show:

BEGIN;

...

ROLLBACK;

Challenge 3 — Atomic RAG Ingestion

Suppose ingestion requires:

1. Insert document

2. Insert chunks

3. Insert ingestion job

4. Mark document as processed

Explain why these operations may need to belong to the **same transaction**.

Then explain one situation where you would deliberately **not** keep all processing inside one transaction.

Challenge 4 — Error State

Consider:

```
BEGIN;
```

```
    INSERT INTO users (  
id,  
name  
)  
    VALUES (  
1,  
'Aarav'  
);
```

```
    INSERT INTO users (  
id,  
name  
)  
    VALUES (  
1,  
'Emma'  
);
```

```
SELECT *  
FROM users;
```

Assume users.id is a primary key.

Questions

1. What happens at the second INSERT?
2. What happens to the transaction?
3. Can the final SELECT execute normally?

4. What should you do?

Challenge 5 — ACID

For each scenario, identify which ACID property is primarily involved.

A

A document and its chunks must either all be inserted or none should be inserted.

B

A foreign key must never reference a nonexistent user.

C

Two concurrent workers shouldn't incorrectly process the same job.

D

After successful commit, data should survive a PostgreSQL crash/restart according to the system's durability guarantees.

Challenge 6 — Isolation Reasoning

Transaction A:

```
BEGIN;
```

```
SELECT page_count  
FROM documents  
WHERE id = 101;
```

Transaction B then:

```
BEGIN;
```

```
UPDATE documents  
SET page_count = 100  
WHERE id = 101;
```

```
COMMIT;
```

Transaction A executes:

```
SELECT page_count  
FROM documents  
WHERE id = 101;
```

Under PostgreSQL's default:

READ COMMITTED

what should you reason about regarding what Transaction A can see?

Then explain how your reasoning changes under:

REPEATABLE READ

Challenge 7 — Lost Update

Suppose:

counter = 10

Two workers do:

Worker A:

read counter → 10

Worker B:

read counter → 10

Worker A:

write 11

Worker B:

write 11

Expected result:

12

but actual result could become:

11

Explain the problem.

Then give **two PostgreSQL-based strategies** for preventing it.

Challenge 8 — FOR UPDATE

Suppose you have:

ingestion_jobs

id | status

---+-----

1 | pending

2 | pending

3 | pending

Two workers are processing jobs.

Write a query that selects one pending job and locks it using:

FOR UPDATE

Then explain what happens if another worker tries to lock the same row.

Challenge 9 — SKIP LOCKED

Now modify Challenge 8 so that workers can skip jobs currently locked by another worker.

Use:

FOR UPDATE SKIP LOCKED

Explain why this pattern is useful for:

RAG ingestion workers

ETL workers

background job processors

Challenge 10 — Deadlock

Consider:

Transaction A:

LOCK user 1

LOCK user 2

Transaction B:

LOCK user 2

LOCK user 1

Explain how a deadlock can happen.

Then propose a design rule that reduces the probability of this problem.

Challenge 11 — Production Architecture

You have this ingestion flow:

API

↓

Insert document

↓

Call embedding API

↓

Wait 8 seconds

↓

Insert embeddings

↓

Mark document processed

A developer puts the entire flow inside one PostgreSQL transaction.

Explain at least **three problems** with this architecture.

Then design a better transaction boundary.

Challenge 12 — Transaction + LEFT JOIN

You are processing users and documents.

Inside a transaction, you run:

SELECT

```
u.id,  
COUNT(d.id)  
FROM users u  
LEFT JOIN documents d  
ON d.user_id = u.id  
AND d.status = 'processed'  
GROUP BY u.id;
```

Then you update users based on these counts.

Explain why the LEFT JOIN semantics still matter **inside a transaction**.

Does being inside a transaction somehow make:

```
LEFT JOIN  
behave like:  
INNER JOIN  
?
```

INTERVIEW ROUND

Answer these without searching.

Q13

What is a database transaction?

Q14

Explain ACID.

Q15

What is the difference between COMMIT and ROLLBACK?

Q16

What happens to a PostgreSQL transaction after a statement error?

Q17

What is the default PostgreSQL transaction isolation level?

Q18

What is the difference between READ COMMITTED and REPEATABLE READ?

Q19

What is SERIALIZABLE isolation?

Q20

What is a dirty read?

Q21

What is a non-repeatable read?

Q22

What is a phantom read?

Q23

What problem does:

SELECT ... FOR UPDATE

solve?

Q24

What is SKIP LOCKED useful for?

Q25

What is a deadlock?

Q26

How can applications reduce deadlock risk?

Q27

Why shouldn't you normally keep a PostgreSQL transaction open while waiting for an external LLM/embedding API?

INTERVIEW SCENARIO

Imagine you're interviewing for a **Data Engineer / GenAI Engineer** role.

The interviewer says:

"We have 20 workers processing RAG ingestion jobs. Sometimes two workers process the same document. Design a PostgreSQL-based solution."

Explain your architecture.

Your answer should consider:

ingestion_jobs

transactions

row locking

FOR UPDATE

SKIP LOCKED

status transitions

COMMIT

worker failure

retry

deadlocks

Don't just give SQL. Explain the **concurrency reasoning**.

Day 18 Mastery Target

By the end of today, you should be able to reason about:

Transaction

↓

BEGIN

↓

Multiple operations

↓

COMMIT / ROLLBACK

↓

ACID

↓

Concurrency

↓

Isolation

↓

Locks

↓

FOR UPDATE

↓

SKIP LOCKED

↓

Deadlocks

And as a production engineer, you should instinctively ask:

"What happens if two workers execute this at the same time?"

That question is one of the biggest differences between writing SQL and engineering database-backed systems.

Your assignment

Solve **Challenges 1–12**, then answer **Interview Q13–27** and the worker-concurrency scenario.

I will review your answers at your current **Advanced SQL → PostgreSQL Engineering** level and specifically test whether you understand **concurrency and transaction reasoning**, rather than rewarding memorized definitions.